RISC — Exam Ready Notes

Microprocessor & Microcontrollers (MPMC) · ARM = **Acorn RISC Machine** · Source: Module 5 "The ARM Instruction Set" (ARM University Program V1.0). Highlighted italic lines are **comments** (why it is asked / how to remember it).

1. What RISC is (definition + origin)

RISC = **Reduced Instruction Set Computer / Computing**. A design philosophy that keeps only a **small set of simple, uniform-length instructions**, each executing in (almost) **one clock cycle**, operating **only on registers**; memory is touched exclusively by explicit **LOAD** and **STORE** instructions.

- **Origin:** early-1980s university research — **Berkeley RISC** (David Patterson) and **Stanford MIPS** (John Hennessy).
- **Idea behind it:** real programs spend most of their time in a small subset of instructions (locality). Make that subset extremely fast; push everything rare into software / the compiler.
- **ARM = Acorn RISC Machine** — developed by **Acorn Computers (UK)**; the official project started **October 1983**, silicon partner **VLSI Technology**, design led by **Wilson and Furber**.
- ARM is a **32-bit RISC load/store architecture**. Memory **cannot** be manipulated directly — data must first be brought into registers.

*Comment: RISC "reduces" the hardware, not the work. Complexity does not disappear — it moves from the chip into the **compiler** and the **programmer**. This one line earns marks in almost every RISC question.*

2. The RISC design rules (feature list)

- **Large, uniform register file** — operands live in registers, not memory.
- **Load/store architecture** — only LDR/STR access memory; ALU instructions never do.
- **Few, simple addressing modes** (immediate, register, register-offset, pre/post-indexed, PC-relative).
- **Fixed-length instructions** → simple, fast, hard-wired decoding.
- **Single-cycle execution** for most instructions → high clock rate.
- **Pipelining** is the main throughput mechanism.
- **Hard-wired control unit** (no micro-code / control store).
- **3-operand register format**: `Rd = Rn op Rm`.
- **Simple data types** — 8 / 16 / 32-bit integers, few formats.
- **Compiler does the heavy lifting** — register allocation, scheduling, expansion of complex operations.

*Comment: every one of these exists for the same reason — to shorten the critical path so the clock can run faster. Learn the list as "**few, fixed, fast, register-only, pipelined, compiler-driven**".*

3. RISC vs CISC (classic 10-mark table)

Parameter	RISC	CISC
Instructions	Small set (~30–100), simple & uniform	Large set (~100–300+), complex, varied
Length	Fixed (ARM: 32 bit)	Variable (1–15 bytes in x86)
Execution	~1 cycle / instruction	Several cycles, varies per instruction
Addressing modes	Few and simple	Many and complex
Memory operand	Only via LOAD / STORE	ALU can operate directly on memory
Registers	Many (large file, banked)	Few
Control unit	Hard-wired	Micro-programmed (control store)
Decoding	Simple, fast	Complex, slow
Code density	Poor (more instructions, larger code)	Good (dense code, less memory)
Compiler role	Heavy dependence	Light dependence
Pipelining	Easy and efficient	Difficult
Chip cost / power	Smaller, cheaper, low power	Larger, costlier, more power
Examples	ARM, RISC-V, MIPS, AVR, PowerPC, SPARC	Intel x86, 8051, Motorola 68k, VAX
Typical use	Mobile, embedded, low-power, SoC	Desktop / server, legacy compatibility

*Comment: bonus line — modern x86 is **CISC outside, RISC inside**: it translates CISC instructions into simple RISC-like micro-ops that are then pipelined.*

4. ARM as a RISC processor — main features of the ARM ISA

- **All instructions are 32 bits long** → uniform fetch and decode.
- **Most instructions execute in a single cycle.**
- **Every instruction can be conditionally executed** (4-bit condition field: EQ, NE, GT, LT, AL…).
- **Load/store architecture** — data-processing instructions act **only on registers**.
- **Three-operand format** (Rd, Rn, Rm).
- **Combined ALU and barrel shifter** → high-speed bit manipulation at no extra cycle cost.
- **Specific memory-access instructions** with powerful **auto-indexing** addressing modes (address updates as you load/store).
- **32-bit and 8-bit data types**, plus **16-bit** from ARM Architecture v4.
- **Flexible multiple-register load/store** — LDM / STM (and PUSH/POP).
- **Instruction-set extension via coprocessors** (CP0–CP15).

Comment: "conditional execution" is ARM's trick to avoid branches — fewer branches means fewer pipeline stalls, which is exactly what a RISC pipeline needs.

Why ARM (RISC) is chosen — advantages

- Cheap compared with other processors; simple design → minimum energy waste.
- Low power consumption → much better battery life (mobiles, tablets, MP3 players, PDAs, hand-held gaming, hard drives).
- Performs **one operation at a time**, hence works faster per operation.
- Wide availability and application support.

ARM vs Intel (RISC vs CISC in the market)

- ARM: very simple structure, minimum energy wastage, low chip price, long history in portable devices.
- Intel: complicated design kept for compatibility with its desktop / laptop CPUs; entered the mobile space later; ARM's growth forced Intel to raise its efficiency.

AVR — another RISC family (module fact)

AVR = **Alf-Egil Bogen & Vegard Wollan RISC** (also called Advanced Virtual RISC), by **Atmel, 1996**. First device **AT90S8515**; first commercial part **AT90S1200 (1997)**. Categories: **TinyAVR** (small, simple apps), **MegaAVR** (up to 256 KB, most popular), **XmegaAVR** (large memory, high speed, complex apps).

PIC (Microchip) also uses **RISC** — **PIC = Harvard / RISC**, **AVR = Harvard / RISC**, whereas **8051** is an 8-bit **CISC-style** family (128 B RAM, 4 KB ROM, 2 timers, 1 serial port, 4 ports on one chip).

5. Register file — RISC means register-rich

- ARM has **37 registers, all 32-bit**: 1 PC + 1 CPSR + **5 SPSRs** + **30 general-purpose** registers.
- **7 processor modes**: **User** (unprivileged, normal tasks), **FIQ** (fast/high-priority interrupt), **IRQ** (normal interrupt), **SVC** (supervisor — reset & SWI), **Abort** (memory access violation), **Undef** (undefined instruction), **System** (privileged, uses User registers).
- **Banking:** the current mode decides which bank is visible. FIQ banks **r8–r14**; IRQ/SVC/Abort/Undef bank only **r13(sp), r14(lr)** (+ SPSR).
- Visible in *every* mode: **r0–r12**, **r13 = sp**, **r14 = lr**, **r15 = pc**, **CPSR**.

Register	Role (must-know)
PC (r15)	Program counter. Any instruction with PC as destination = a **branch**.
LR (r14)	Link register. Holds a copy of PC on **BL** (subroutine call) or on an exception; copied back to PC on return.
SP (r13)	Stack pointer. There is **no stack in the architecture** — the stack is program-managed.
CPSR	Current Program Status Register: flags **N Z C V** + mode bits.
SPSR	Saved CPSR while an exception/interrupt runs; restored on return. **None in User or System mode.**

Comment: banked registers are a pure RISC performance choice — on a mode switch the hardware gives the interrupt its own sp/lr, so nothing has to be saved first → very fast interrupts. FIQ gets the most private registers (r8–r14) because it must be the fastest.

6. Data-flow model — how load/store actually happens

- Sources **Rn** and **Rm** are read from the register file on internal buses **A** and **B**; the result is written back to **Rd**.
- The **ALU** or **MAC** (multiply-accumulate) unit computes the result.
- **Rm may first pass through the barrel shifter** (LSL, LSR, ASR, ROR). Barrel shifter and ALU work together **in the same cycle**, so a shifted operand and an address calculation cost nothing extra.
- **Load/store:** the ALU is used to **compute the address**, which is held in the address register and broadcast on the address bus.

- **Sign-extend hardware** widens signed 8/16-bit values to 32 bits as they are loaded; unsigned values are zero-filled.
- **Incrementer:** PC goes into the address register, is incremented, and the new value is copied back into r15 and used for the next fetch.
- **Von Neumann** core (ARM7): one shared bus carries *either* instruction *or* data; the instruction decoder translates the instruction before execution.

*Comment: the sentence to quote in an exam — "there are **no data-processing instructions that directly manipulate data in memory**; data items are placed in the register file." That is the definition of a load/store (RISC) architecture.*

7. Pipelining — the RISC throughput trick

The pipeline lets several operations overlap instead of running serially, raising throughput without a faster clock.

ARM7: 3-stage pipeline

- **Fetch** — read the opcode from memory into the internal instruction register.
- **Decode** — activate the control lines for that opcode; read register operands (the file has **3 read ports**, so most instructions get all operands in one cycle).
- **Execute** — shift + ALU operation; for a load/store the ALU computes the memory address.

Throughput: simple instructions such as **ADD** complete at a rate of **one per cycle**. More complex ones cost extra — **STR needs 2 effective clock cycles (+1 cycle)** for the memory access.

	c1	c2	c3	c4	c5
Instr 1 (at PC)	F	D	E		
Instr 2 (at PC+4)		F	D	E	
Instr 3 (at PC+8)			F	D	E

At cycle c3: executing = instruction at PC, decoding = PC+4, fetching = PC+8 ⇒ the stored PC value reads **8 bytes ahead** of the executing instruction.

Key fact: the PC points to the instruction being **fetched**, not the one being executed. In ARM state, therefore, **`PC = address of current instruction + 8`** (two words ahead): executing = PC–8, decode = PC–4, fetch = PC.

Comment: "Why is PC = PC+8 in ARM?" is a favourite 2/5-mark question. Answer: 3-stage pipeline → fetch is 2 stages ahead of execute, and ARM instructions are 4 bytes, so 2 × 4 = 8. (In Thumb state, instructions are 2 bytes → PC+4.)

ARM9: 5-stage pipeline

Fetch → **Decode** → **Execute** → **Buffer/Data** (access data memory, or simply buffer the ALU result for one cycle) → **Write back** (result, including loaded data, returned to the register file).

Item	ARM7	ARM9
Architecture	Von Neumann (shared bus)	Harvard (separate I & D memory)
Pipelining	3-stage	5-stage
ISA	v4T	v4T

Comment: the 5-stage pipeline is only possible because Harvard architecture has separate instruction and data memories — an instruction fetch and a data access can then happen in the same cycle. Link ARM7/ARM9 → Von Neumann/Harvard → 3/5-stage; they are always asked together.

Pipeline hazards and their fixes

- **Structural hazard** — two instructions need the same resource in the same cycle. *Fix:* separate instruction and data memories / caches (I$ and D$); for register-file ports, **read in the second half** of the cycle and **write in the first half**.
- **Data hazard** — an instruction needs a result still inside the pipeline (classic `lw` followed by `add`). *Fix:* stall the pipeline, or forward/bypass the result.
- **Control hazard** — a branch must decide the next PC before the condition is evaluated and the target computed. *Fix:* stall (this hurts CPI), branch prediction, or ARM's conditional execution.

Cost formula: CPU time = instruction count × CPI × clock period. Every stall raises CPI.

8. Advantages and limitations of RISC

Advantages	Limitations
Simple hardware → smaller, cheaper, lower-power chip	Lower code density → more instructions, more program memory
Fixed-length, single-cycle instructions → high clock, easy pipelining	Heavy dependence on a good compiler
Regularity → excellent compiler optimisation	Simple loops need extra load/store instructions
Many registers → less memory traffic	Pipeline hazards / stalls reduce the theoretical 1 CPI
Predictable timing → ideal for embedded & real-time	Poor fit for complex legacy (CISC) software

9. One-minute revision block

Mnemonic — R.I.S.C.: Registers many · **I**nstructions few and fixed-length · **S**ingle-cycle and load/Store only · **C**ompiler does the rest.

Numbers to memorise: 32-bit instructions · 37 registers / 30 GPRs · 7 modes · 5 SPSRs · 3 read ports · ARM7 = 3-stage, ARM9 = 5-stage · PC+8 · STR = 2 cycles · ARM project Oct 1983 · AVR 1996.

Likely questions: (1) Define RISC and explain its features. (2) Compare RISC with CISC. (3) Explain why ARM is a RISC processor / list the main features of the ARM instruction set. (4) Describe ARM register organisation and banking. (5) Explain the ARM pipeline and why PC = PC+8. (6) Explain pipeline hazards with remedies. (7) Distinguish ARM7 and ARM9.

Comment: for 10-mark answers always draw the 3-stage pipeline diagram (F – D – E with PC, PC–4, PC–8) or the RISC-vs-CISC table — a diagram or table is what separates a 6 from a 9.